

OSU Satellite TV for MITRE eCTF 2025

Contents

1. Team	1
2. Introduction	2
2.1. Functional Requirements	2
2.1.1. Functional Requirement 1: List Channels	3
2.1.2. Functional Requirement 2: Update Subscription	3
2.1.3. Functional Requirement 3: Decode Frame	3
2.2. Security Requirements	3
2.2.1. Security Requirement 1: Active Subscription	3
2.2.2. Security Requirement 2: Authentication	3
2.2.3. Security Requirement 3: Increasing Timestamps	3
2.3. Attack Scenarios	3
2.3.1. Attack Scenario 1: Expired Subscription	3
2.3.2. Attack Scenario 2: Pirated Subscription	4
2.3.3. Attack Scenario 3: No Subscription	4
2.3.4. Attack Scenario 4: Recording Playback	4
2.3.5. Attack Scenario 5: Pesky Neighbor	4
3. Deployment	4
3.1. Secrets	4
3.2. Decoder	4
4. System Design	4
4.1. Encode and Decode	4
4.2. Subscription	5
4.3. Signatures	6
4.4. Global Timestamp	6
4.5. Zig	6
5. Appendix	6
5.1. Proof that there are no more than 126 nodes	6
5.1.1. Definitions	6
5.1.2. Lemma 1	6
5.1.3. Lemma 2	7
5.1.4. Theorem	7
5.1.5. Application	8
5.2. Proof of node partitioning optimality	8
5.2.1. Correctness	8
5.2.2. Optimality	8

1. Team

The team is comprised of the following people from the Ohio State State University:

Student	Advisor
Jack Leverett	Dr. Felix Engelmann

Student	Advisor
Divij Sasidhar	Julia Armstrong
Mark Bundschuh	
Maxwell McGee	
Matthew Weikel	
Debjani Hill	
Aditya Gupta	
Jacob Mcquade	
Eva Smith	
William Comer	
Diego Noria	
Zihao Zhang	
Mason Erwine	
Christopher Begines	
Haoling Zhou	
Ryan Jackman	
Joshua Sims	
Rachel Thoi	
Cadan Keller	
James Vendetti	

2. Introduction

This document describes OSU's implementation of a secure satellite TV system as specified by MITRE's eCTF 2025. The Satellite TV system features a simulated satellite sending ASCII art to subscribed hardware Decoder devices, implementing Functional Requirements 2.1. The decoder devices are MAX78000FTHR microcontrollers by Analog Devices. The system is secured according to the Security Requirements 2.2, and designed to defend against attacks as outlined in Attack Scenarios 2.3, which revolve around erroneously decoding frames sent by a satellite.

2.1. Functional Requirements

The two main components are the Encoder and Decoder. The encoder receives frames of up to 64 bytes, and converts them into our custom format before sending it over the satellite link. Then, each decoder receives the frame, and must decode the message into the original 64 bytes.

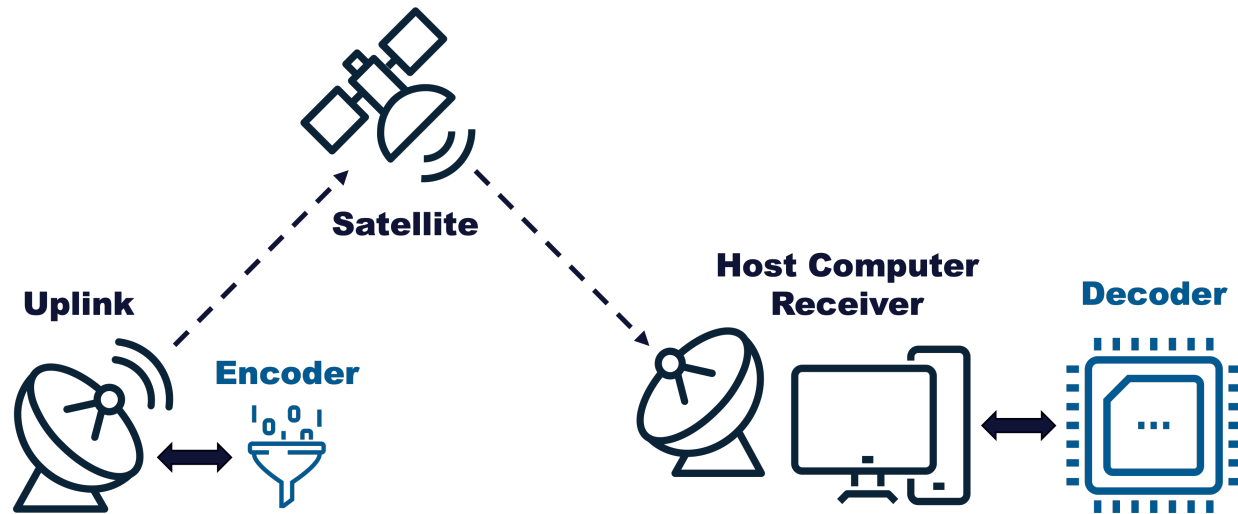


Figure 1: Full system with blue highlights indicating components which need to be implemented.

2.1.1. Functional Requirement 1: List Channels

“The Decoder must be able to list the channel numbers that the Decoder has a valid subscription for.”

2.1.2. Functional Requirement 2: Update Subscription

“The Decoder must be able to update its channel subscriptions with a valid update package. If a subscription update for a channel is received for a channel that already has a subscription, the Decoder must only use the latest subscription update.”

2.1.3. Functional Requirement 3: Decode Frame

“The Decoder must properly decode TV frame data from any channel that has a valid and active subscription installed or for any emergency broadcast frames.”

2.2. Security Requirements

2.2.1. Security Requirement 1: Active Subscription

“An attacker should not be able to decode TV frames without a Decoder that has a valid, active subscription to that channel.”

2.2.2. Security Requirement 2: Authentication

“The Decoder should only decode valid TV frames generated by the Satellite System the Decoder was provisioned for.”

2.2.3. Security Requirement 3: Increasing Timestamps

“The Decoder should only decode frames with strictly monotonically increasing timestamps.”

2.3. Attack Scenarios

We consider 5 different scenarios where the attackers have access to certain information that the system is designed to protect against.

2.3.1. Attack Scenario 1: Expired Subscription

An attacker has physical access to a decoder with a valid subscription to a channel. The timestamp of the current frames sent by the encoder is past the ending timestamp of the attacker’s subscription.

every timestamp. Figure 2 shows a hash key derivation tree for $t \in [0, 7]$ and has 8 leaves. Note that the full sized tree has 2^{64} leaves, the leaves are computed on demand for particular timestamps since it would be computationally infeasible to precompute all keys for such a large tree.

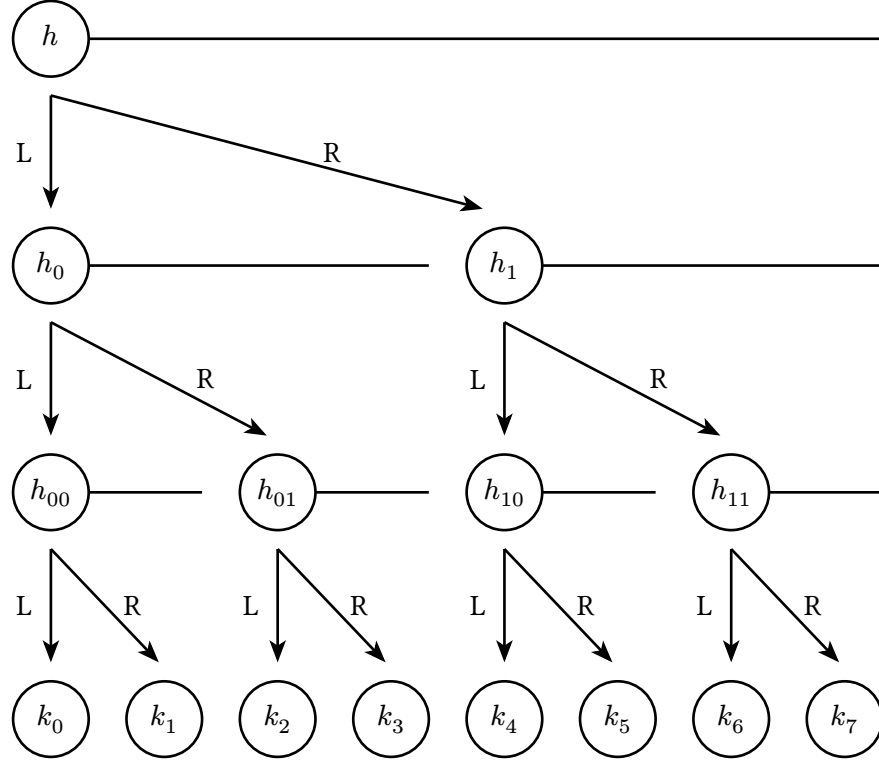


Figure 2: A hash key derivation tree from $t = 0$ to $t = 7$

To send the keys to the decoder, they are compressed into a subscription file. The compression is done by finding the largest power of two intervals which span the entire interval of timestamps $t \in [\text{start}, \text{end}]$. For example, all keys in $[1, 5]$ can be derived by $h_{001} \Rightarrow \{k_1\}$, $h_{01} \Rightarrow \{k_2, k_3\}$, and $h_{10} \Rightarrow \{k_4, k_5\}$, so only h_{001} , h_{01} , and h_{10} are needed to form a subscription. The worst case interval is $[1, 2^{64} - 2]$ which requires 126 subtree roots.

- **SR1**
- **AS1:** Expired Subscription
- **AS3:** No Subscription
- **AS4:** Recording Playback

4.2. Subscription

Field Name	Size	Relative Size
Channel ID	1 byte	■
Start Timestamp	8 bytes	■■■■■■■■
End Timestamp	8 bytes	■■■■■■■■
Packed Subtree Root Hashes	$\{16n \text{ bytes} \mid n \in \mathbb{N}, 1 \leq n \leq 126\}$	■■■■■■■■■■■■■■■■■■■■ ... n times

The decoder runs the same algorithm using the start and end timestamps as the encoder to figure out where the subtree root hashes are, so no extra metadata is required to positions the hashes, or to know how many there are.

The entire subscription payload is symmetrically encrypted using Salsa20 with a key $k_{\text{subscription}} = H(S + \text{DEVICE_ID})$. $k_{\text{subscription}}$ is stored within the decoder when it is provisioned and can be used to decrypt a subscription. This ensures that subscriptions can only be used by the decoder the subscription was generated for.

- **AS2:** Pirated Subscription

4.3. Signatures

The encoder uses the secret key SK to sign every message with a 64 byte signature. The public key PK is stored within all decoders and verifies every message to ensure it came from the encoder it was provisioned with.

- **SR2**
- **AS5:** Pesky Neighbor

4.4. Global Timestamp

A global timestamp is kept in the RAM of the decoder, and is used to reject misordered frames. It increases on every successful decode.

- **SR3**

4.5. Zig

The Zig programming language was selected as the language to write the secure decoder in. Zig has memory safety features like protection from indexing an array out of bounds, double free protections and more. Additionally, it includes a comprehensive standard library which securely implements many cryptographic functions, among many other quality of life functions. Most importantly, Zig can do all this while maintaining perfect C compatibility and using the MSDK provided by Analog Devices Inc (the developer of the MAX78000FTHR), without requiring a rewrite of the Hardware Abstraction Layer (HAL). This allowed the team to move incredibly quickly while ensuring a high level of security.

5. Appendix

5.1. Proof that there are no more than 126 nodes

5.1.1. Definitions

For a given n , a *node* has an offset o where $o \in \mathbb{N}$ and $0 \leq o \leq 2^n - 1$, and a length $l \in \{2^x \mid x \in \mathbb{N} \wedge 0 \leq x \leq n\}$.

To say that an interval I can be *encoded* by n nodes is to say that, $\bigcup_i^n [o_i, o_i + l_i - 1] = I$.

Two intervals, $I_1 = [o_1, o_1 + l_1 - 1]$, and $I_2 = [o_2, o_2 + l_2 - 1]$ are *analogous* if $l_1 = l_2$ and $o_1 \bmod l_1 = o_2 \bmod l_2$.

5.1.2. Lemma 1

Any interval on $[0, 2^n - 1]$ where $n \in \mathbb{N}, n \geq 1$ and starting at 0 can be encoded with at most n nodes.

Base Case, $n = 1$:

Let I be an interval on $[0, 2^1 - 1] = [0, 1]$ starting at 0. For $I = [0, 0]$, this can be covered by a 1 length node at offset 0. For $I = [0, 1]$, this can be covered by a 2 length node at offset 0.

Induction Hypothesis: Any interval on $[0, 2^n - 1]$ and starting at 0 can be encoded with at most n nodes.

We wish to show that any interval on $[0, 2^{n+1} - 1]$ and starting at 0 can be encoded with at most $n + 1$ nodes.

Let I be an interval in $[0, 2^{n+1} - 1]$ starting at 0, i.e. $I = [0, k]$ where $k < 2^{n+1}$.

Case 1: $k < 2^n$: By the induction hypothesis, I can be covered by $n \leq n + 1$ nodes.

Case 2: $k \geq 2^n$: Then $I = [0, 2^n - 1] \cup [2^n, k]$. The interval $[0, 2^n - 1]$ can be covered by 1 node, because it is half of the total interval. $[2^n, k]$ is analogous to $[0, 2^n - 1]$. By the induction hypothesis, $[0, 2^n - 1]$ can be covered by at most n nodes. Therefore, $[2^n, k]$ can be covered by at most n nodes. Therefore, I can be covered by at most $n + 1$ nodes.

5.1.3. Lemma 2

Any interval on $[k, 2^n - 1]$ where $n \in \mathbb{N}, n \geq 1$ and starting at k can be encoded with at most n nodes.

This follows from Lemma 1, because the interval $[k, 2^n - 1]$ is analogous to $[0, 2^n - 1]$.

5.1.4. Theorem

Any interval on $[0, 2^n - 1]$ where $n \in \mathbb{N}, n \geq 2$ can be covered by at most $2n - 2$ nodes.

Base Case, $n = 2$:

Let I be an interval on $[0, 2^2 - 1] = [0, 3]$.

Induction Hypothesis: Any interval on $[0, 2^n - 1]$ where $n \in \mathbb{N}, n \geq 2$ can be covered by at most $2n - 2$ nodes.

We wish to show that any interval on $[0, 2^{n+1} - 1]$ where $n \in \mathbb{N}, n + 1 \geq 2 \Rightarrow n \geq 1$ can be covered by at most $2(n + 1) - 1 = 2n$ nodes.

Let I be an interval on $[0, 2^{n+1} - 1]$.

Case 1: $I \subset [0, 2^n - 1]$. Intuitively, this is within the left half of the interval. By the induction hypothesis, I can be covered by $2n - 2 \leq 2n$ nodes.

Case 2: $I \subset [2^n, 2^{n+1} - 1]$. Intuitively, this is within the right half of the interval. The interval $[2^n, 2^{n+1} - 1]$ is analogous to the interval $[0, 2^n - 1]$. By the induction hypothesis, $[0, 2^n - 1]$ can be covered by $2n - 2$ nodes. Therefore, I can be covered by $2n - 2 \leq 2n$ nodes.

Case 3: $k < 2^n < l$. Intuitively, this means the interval crosses the middle boundary. Then, $I = [k, 2^n - 1] \cup [2^n, l]$. By Lemma 2: 5.1.3, $[k, 2^n - 1]$ can be covered by at most n nodes. Similarly, $[2^n, l]$ is analogous to $[0, l - 2^n]$, which, by Lemma 1: 5.1.2, can be covered by at most n nodes. Therefore, I can be covered by $n + n = 2n$ nodes.

5.1.5. Application

Timestamps are 64 bit unsigned integers, so we choose $n = 64$. Theorem 5.1.4 shows that no more than $2(64) - 2 = 126$ nodes are required to encode any interval from $[0, 2^{64} - 1]$.

5.2. Proof of node partitioning optimality

We will show that the following algorithm correctly produces the optimal set of nodes to cover an interval $[a, b]$.

```
1 class Node:
2     offset: int
3     length: int
4
5 def get_nodes(a: int, b: int) -> list[Node]:
6     nodes = []
7
8     while a <= b:
9         power = 1
10        while (a + (1 << power) - 1) <= b and power <= ctz(a):
11            power += 1
12        power -= 1
13
14        end = a + (1 << power) - 1
15        nodes.append(Node(length=2**power, offset=a))
16
17        a = end + 1
18
19    return nodes
```

5.2.1. Correctness

5.2.2. Optimality